**Software Development Project — G&G City · Lab Backlog Sprint 1**

**LAB C3: The Box + The Command Box — Mohammed Al-Kathiri**

---

Topic: State-Driven SVG + Validating User Input Against Data

## Learning outcomes

### By the end of this lab, students will be able to:

- Show and hide SVG layers from a single `state` value
- Normalize text input and compare it with rules from a JSON file
- Lock a form while an async action is running

## 🎯 Problem Scenario

**The learner types commands in a terminal-like box. Your code decides if the command is right, and your box changes when it is.**

- **Read `data/lessons.json` (owned by P1, do not edit it): `accepted`, `commonMistakes`, `boxStateBefore/After`, `matching`**
- **Your command box only reports the result with `onResult`. It never moves the cat (S3 does that)**

**Your files: `box/box.svg, box/box.js, box/test.html`**

## Required Outputs

### Build:

1. **`box.svg` with layers that appear per state: `empty` (closed lid) and `initialized` (open lid + `.git` sticker)**
2. **`createBox(container, svgUrl)` → `{ el, ready, setState(name), setLabel(text) }`**
3. **`checkCommand(input, lesson, matching)` → `{ ok, feedback }`**
4. **`createCommandBox(container, lesson, onResult, matching)` that locks itself while `onResult` runs**
5. **`test.html` that tests right and wrong inputs**

Code is given only for the hard parts. Parts marked 📝 **Note** are easy: follow the note and write them yourself. Test with **Live Server** in VS Code (`fetch` does not work when you double-click a file).

## Model Answer:

### Task 1: Draw the box

> 📝 **Note (no code needed):** Draw it like C1 drew the cat. Layers that are always visible have no extra attribute. Layers that depend on the state get `data-show="..."` with the states they appear in, e.g. `<g id="box-lid-open" data-show="initialized">`. Put a `<text id="box-label">` on the sticker for `setLabel`.

### Task 2: createBox

> 📝 **Note (no code needed):** Same pattern as `createCat`: a wrapper `div.gg-box`, `fetch` the SVG, put it inline, return an object. Remember the current state and label in variables so `setState` works even before the SVG has loaded.

The only tricky part is the render function:

```
// Every layer with data-show="a b" is visible only when the state is a or b
function render() {
  el.dataset.state = state;
  el.querySelectorAll('[data-show]').forEach(layer => {
    layer.style.display = layer.dataset.show.split(' ').includes(state) ? '' : 'none';
  });
  const text = el.querySelector('#box-label');
  if (text) text.textContent = label;
}
```

### Task 3: Checking the command (hard part)

Both sides are normalized the same way, so `' git init '` matches `'git init'`. Common mistakes are checked before the generic failure message.

```
const DEFAULT_MATCHING = { trim: true, collapseSpaces: true, caseSensitive: true };

function normalize(text, m) {
  let s = String(text);
  if (m.trim)           s = s.trim();
  if (m.collapseSpaces) s = s.replace(/\s+/g, ' ');
  if (!m.caseSensitive) s = s.toLowerCase();
  return s;
}

function checkCommand(input, lesson, matching = DEFAULT_MATCHING) {
  const n = t => normalize(t, matching);
  const typed = n(input);

  if (!typed)                                    return { ok: false, feedback:
```

```
lesson.cat.hint };
  if (lesson.accepted.some(a => n(a) === typed)) return { ok: true,  feedback:
lesson.cat.success };

  const mistake = (lesson.commonMistakes || []).find(m => n(m.input) === typed);
  return { ok: false, feedback: mistake ? mistake.feedback : lesson.cat.failure };
```

### Task 4: The command box (hard part)

`await onResult(result)` is the key line: the input stays disabled until the cat has finished its spell.

```
function createCommandBox(container, lesson, onResult, matching) {
  const form = document.createElement('form');
  form.className = 'gg-command';
  form.dir = 'ltr';                                    // commands are always left-to-
right
  form.innerHTML = `
    <span class="gg-prompt">$</span>
    <input type="text" autocomplete="off" spellcheck="false" aria-label="Command">
    <button type="submit">Run</button>`;
  container.appendChild(form);
  const input = form.querySelector('input');

  form.addEventListener('submit', async e => {
    e.preventDefault();                                // no page reload
    if (input.disabled) return;
    const result = checkCommand(input.value, lesson, matching);
    input.disabled = true;                             // block double submits during the
spell
    await onResult(result);                            // S3 plays charge → spell → say
    if (!result.ok) { input.disabled = false; input.select(); }
  });

  return { el: form, focus: () => input.focus() };
```

### Task 5: test.html

> 📝 **Note (no code needed):** Load `../data/lessons.json` with `fetch`, take `stations[0].lessons[0]`, and log `checkCommand` for: `git init`, `' git init '`, `git init .`, `Git init`, `git int` and an empty string. Then add the command box and a box, and call `box.setState(lesson.boxStateAfter)` when `ok` is true.

## Expected Output




Git init → locked while the cat reacts




git init → box opens + .git

## Submission

```
git pull
git add box/
git commit -m "إضافة الصندوق ومربع الأمر"
git push
```